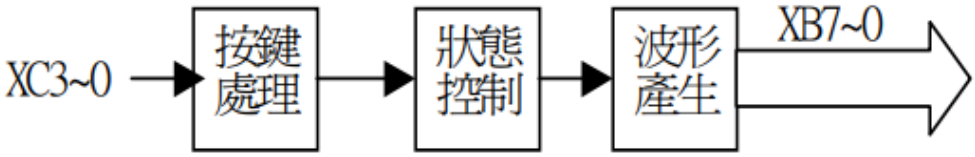
FPGA 數位邏輯系統設計 作業九

第九章 程序處理

通訊 4A 0106C033 桑瑋佑

一 實驗目的:

結合按鍵處理、狀態控制和波形顯示功能，建立一個四鍵輸入的電子密碼鎖電路。



二 步驟:(雙)

實驗時，先用 LoadPGA 程式將 ch09.rbt 的電路檔載入 FPGA 中，試著依定義來控制按鍵，再由 LED 的顯示狀況去確認 FPGA 是否正確執行。由於密碼定義為，最好能敲入，用多出的兩碼來確定讓密碼鎖復歸到 S0 的狀態。

按下 XC(0~3)來輸入設定好密碼，當密碼正確時，執行跑馬燈。

三 結果:以下為程式碼，為 Ch09.vhd 檔

```
--The IEEE standard 1164 package, declares std_logic, rising_edge(), etc.
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
```

```
--
--*******************************************************
-- CH09 : practise STATE MACHINE and SHIFT REGISTER
--                              designed by Pei-Chong Tang, Jan. 1999
--*******************************************************
--
entity CH09 is
        port
        (       CLK : in    STD_LOGIC;
                XA    : out STD_LOGIC_VECTOR (3 downto 0);
                XB    : out STD_LOGIC_VECTOR (7 downto 0);
                XC    : in    STD_LOGIC_VECTOR (3 downto 0)
        );
end CH09;

architecture CH09_ARCH of CH09 is
        signal SAMPLE,DISPLAY      : STD_LOGIC;
        signal KEY,FLT,DIF,DOUT : STD_LOGIC;
        signal DIN : STD_LOGIC_VECTOR (1 downto 0);
begin
--
--*******************************************************
--
-- 20-BIT free counter
--
FREE_COUNTER : block
        signal Q          : STD_LOGIC_VECTOR (19 downto 0);
        signal D1,D2 : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        D1 <= Q(16);                        --delay
                        D2 <= Q(19);                        --delay
                        Q      <= Q+1;                        --UP counter
                end if;
        end process;
        SAMPLE    <= Q(16) and not D1;                --differential (125Hz)
```

```
        DISPLAY <= Q(19) and not D2;                --differential ( 16Hz)
end block FREE_COUNTER;
--
--*****************************************************
--
-- keyboard decoding
--
KEYBOARD_DECODING : block
begin
        KEY <= not(XC(0) and XC(1) and XC(2) and XC(3));    --any keyin
        DIN <= "00" when XC="1110" else                         --<0> for XC(0)
                "01" when XC="1101" else                        --<1> for XC(1)
                "10" when XC="1011" else                        --<2> for XC(2)
                "11";                                            --<3> for XC(3)
end block KEYBOARD_DECODING;
--
--*****************************************************
--
-- keyboard debouncing
--
DEBOUNCING : block
        signal D0,D1 : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then       --CLK rising
                        if SAMPLE='1' then                --sampling control
                                D1<=D0; D0<=KEY;            --delay
                                FLT <= ((D0 and D1) or FLT) and (D0 or D1); --RS-F/F
                        end if;
                end if;
        end process;
end block DEBOUNCING;
--
--*****************************************************
--
-- differential signal
--
```

```
DIFFERENTIAL : block
        signal D0,D1 : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        D1<=D0; D0<=FLT;                    --delay
                end if;
        end process;
        DIF <= D0 and not D1;                          --differential
end block DIFFERENTIAL;
--
--*******************************************************
--
-- state machine
--
STATE_MACHINE : block
        type STATE is (S0,S1,S2,S3,S4,S5,S6,S7,S8); --STATE definition
--        attribute enum_encoding of
--        STATE : type is "0000 0001 0010 0011 0100 0101 0110 0111 1000";
        signal EC           : STD_LOGIC;
        signal NOW,NXT : STATE;
begin
        process (CLK)                                          --sequential logic
        begin
                if CLK'event and CLK='1' then             --CLK rising
                        if EC='1' then                         --CLK enable
                                NOW <= NXT;                         --state to next
                        end if;
                end if;
        end process;
        process (NOW,DIN)                                      --combinational logic
        begin
                case NOW is
                when S0 =>                                     --in state S0
                        DOUT <= '0';
                        if DIN="11" then                       --check <1>
                                NXT <= S1;
```

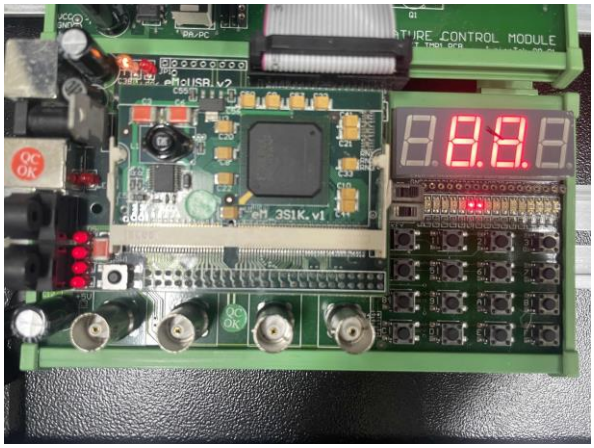
```
        else
                NXT <= S0;
        end if;
when S1 =>                                      --in state S1
        DOUT <= '0';
        if DIN="01" then                        --check <2>
                NXT <= S2;
        else
                NXT <= S0;
        end if;
when S2 =>                                      --in state S2
        DOUT <= '0';
        if DIN="10" then                        --check <3>
                NXT <= S3;
        else
                NXT <= S0;
        end if;
when S3 =>                                      --in state S3
        DOUT <= '0';
        if DIN="00" then                        --check <3>
                NXT <= S4;
        else
                NXT <= S0;
        end if;
when S4 =>                                      --in state S4
        DOUT <= '0';
        if DIN="11" then                        --check <0>
                NXT <= S5;
        else
                NXT <= S0;
        end if;
when S5 =>                                      --in state S5
        DOUT <= '0';
        if DIN="00" then                        --check <1>
                NXT <= S6;
        else
                NXT <= S0;
        end if;
```

```
            when S6 =>                                  --in state S5
                DOUT <= '0';
                if DIN="01" then                        --check <1>
                    NXT <= S7;
                else
                    NXT <= S0;
                end if;
            when S7 =>                                  --in state S6
                DOUT <= '1';                            --signal OK
                NXT    <= S0;
            when others =>                              --in others
                DOUT <= '0';
                NXT    <= S0;
            end case;
        end process;
        EC <= DIF;                                      --sample signal
        XA <= "1111" when NOW=S0 else                   --state monitor
              "1110" when NOW=S1 else
              "1101" when NOW=S2 else
              "1100" when NOW=S3 else
              "1011" when NOW=S4 else
              "1010" when NOW=S5 else
              "1001" when NOW=S6 else
              "1111";
end block STATE_MACHINE;
--
--*******************************************************
--
-- waveform-generation by shift-register
--
SHIFT_REGISTER : block
        signal STS       : STD_LOGIC_VECTOR (7 downto 0);
        signal RST,EC : STD_LOGIC;
begin
        process (CLK,RST)
        begin
            if RST='1' then                             --reset status
```

```
                STS <= "00000001";                  --to S0
            elsif CLK'event and CLK='1' then    --CLK rising
                if EC='1' then
                    STS <= STS(6 downto 0) & STS(7); --rotate left
                end if;
            end if;
        end process;
        RST <= not DOUT;                                --RST signal
        EC    <= DOUT and DISPLAY;                        --EC    signal
        XB(0) <= not(DOUT and (STS(0) or STS(7))); --LED output
        XB(1) <= not(DOUT and (STS(1) or STS(6)));
        XB(2) <= not(DOUT and (STS(2) or STS(5)));
        XB(3) <= not(DOUT and (STS(3) or STS(4)));
        XB(4) <= not(DOUT and (STS(4) or STS(3)));
        XB(5) <= not(DOUT and (STS(5) or STS(2)));
        XB(6) <= not(DOUT and (STS(6) or STS(1)));
        XB(7) <= not(DOUT and (STS(7) or STS(0)));
end block SHIFT_REGISTER;
--
--*****************************************************
-- end of architechture
--
end CH09_ARCH;
```

以下為成果圖:

按下 XC(0~3)輸入密碼，單號密碼為 3120301，輸入正確時才會跑出波形，否則須重新輸入密碼。



當密碼輸入正確時回執行設定好的波形，從兩側往內亮，再從中間往外亮。

四 重點提醒:

1. when-else 指令適用於並行模式，僅允許一個輸出訊號；而 case-when 指令則用於序列模式，允許有多個輸出訊號。

2. 可以透過 type 指令來定義名稱，從而提升程式的可讀性。

3. 狀態編碼有多種方法。二進位編碼使用起來簡單且直觀，但可能導致多位元同時變化，若輸出應用於非同步邏輯，容易引發許多非預期的雜訊。而 Gray 編碼的特點是每次僅有一位元變化，因此不會產生雜訊干擾，但其缺點在於：不易記憶，容易產生誤解，且僅適用於狀態按順序改變的情況，對於隨機組合的狀態變化無法發揮作用。

五 心得結論:

這次學習了一個新的語法，雖然程式碼看起來變長了，但可讀性提升了不少，對照著狀態圖就能直接寫出來。而透過使用 type 語法，整體的可讀性更進一步提高。隨著學習到越來越多的語法，熟

練運用它們變得非常重要。同時，之前學過的語法也不能忽略，因為它們會頻繁地出現在後續的程式中。

上課時即使對新內容稍有理解，也不能因此放鬆，必須保持專注，否則像小考這樣難度不高的題目，也可能因為一時不注意而失分。

在面對越來越長的程式碼時，我開始嘗試將程式碼劃分成區塊，分別理解每個區塊的目標，然後逐步讀懂。這樣的方式能降低對長程式碼的畏懼，並幫助我回顧與相關部分的課程內容。熟練後，我相信自己可以更好地掌握這門課程的內容。